

Services, Repositories, Validation and Multiple Users

Lecture 08 | Backend Development with C#

Build a layered, persistent console application where each user can see only their own expenses.

Milestone 03: Layered Multi-user Console Application

- Register a user without saving their password in plain text.
- Log in and remember the current user for the console session.
- Add, list, total and delete expenses only for that user.
- Persist users and expenses in separate JSON files.
- Keep rules in services, file details in repositories, and prompts in Presentation.



Problem / risk

One-user version: every expense in one list.

Question: where should the rule 'Bob cannot remove Alice's expense' live?

Answer: in the application boundary, enforced by repository queries that include UserId.

Better boundary

Multi-user version: every expense has an owner.

- Registration, input conversion, validation, JSON, menus and security have different reasons to change.
- Mixing them makes code difficult to test and dangerous to copy.
- A layer is a responsibility boundary, not an extra ceremony.
- The payoff: replace JSON with SQL later without rewriting login and expense rules.

Lecture08_Services_Repositories_Validation_and_Multiple_Users

Layers

Domain	User and Expense data
Application	service rules and repository contracts
Infrastructure	JSON files and PBKDF2 hashing
Presentation	console menus and input conversion

Lecture08.CodeExamples

Project composition root and executable app

Domain	Business nouns and their properties	Console, JSON, cryptography
Application	Use cases, validation, interfaces	File.ReadAllText, menu loops
Infrastructure	JSON and PBKDF2 implementations	Menu decisions
Presentation	Questions, TryParse, output	Persistence and rules
Project	Wiring concrete implementations	Duplicate business logic

Presentation -> Application -> Domain

Infrastructure -> Application + Domain

Project references Application + Infrastructure + Presentation

Domain references nothing in this solution.


```
public class User
{
    public int Id { get; set; }
    public string Username { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

```
public class Expense
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public string Category { get; set; } = string.Empty;
}
```

User: Id 1, Username alice, PasswordHash ...

User: Id 2, Username bob, PasswordHash ...

Expense: Id 1, UserId 1, Lunch, 12.50

Expense: Id 2, UserId 1, Train, 4.20

Expense: Id 3, UserId 2, Book, 19.00

```
public interface IUserRepository
{
    List<User> GetAll();
    User? GetByUsername(string username);
    void Add(User user);
}

public interface IExpenseRepository
{
    List<Expense> GetByUserId(int userId);
    bool DeleteByIdForUser(int expenseId, int userId);
}
```

Problem / risk

Risky: service receives every expense, then hopes to filter correctly.

*DeleteByIdForUser checks both conditions in one operation.
This is defense in depth: the service and repository both express ownership.*

Better boundary

Focused: repository receives userId and returns only matching records.

- AuthService speaks about Register and Login.
- ExpenseService speaks about AddForUser, GetForUser and DeleteForUser.
- Services coordinate domain data and repository contracts.
- Services return a result and a clear message; Presentation decides how to print it.
- Services do not call `Console.ReadLine` or `File.ReadAllText`.

1 Presentation reads username and password.

V

2 AuthService trims input and validates its rules.

V

3 AuthService asks IUserRepository whether the username is already used.

V

4 IPasswordService creates a protected record.

V

5 AuthService creates User and calls users.Add(user).

V

6 JsonUserRepository serializes the user list.

```
username = username?.Trim() ?? string.Empty;
password ??= string.Empty;

if (username.Length < 3)
{
    message = "Username must contain at least 3 characters.";
    return false;
}
if (password.Length < 8)
{
    message = "Password must contain at least 8 characters.";
    return false;
}
```


Presentation	Can raw text become decimal or int?	TryParse
Application	Is this value acceptable for the use case?	amount > 0
Repository	Can the operation be stored or queried?	JSON read and write
Domain	What does the data represent?	Expense.UserId

```
if (currentUser is null) return false;
if (name.Length < 2) return false;
if (amount <= 0) return false;
if (category.Length == 0) return false;

expenses.Add(new Expense
{
    Id = nextId,
    UserId = currentUser.Id,
    Name = name, Amount = amount, Category = category
});
```

```
public bool DeleteForUser(User? currentUser, int expenseId,
    out string message)
{
    if (currentUser is null) return false;

    if (expenses.DeleteByIdForUser(expenseId, currentUser.Id))
        return true;

    message = "Expense was not found in your account.";
    return false;
}
```

Problem / risk

Never store: password = summer2026

Do not use one fast SHA-256 hash by itself for user passwords.

Do not use one global fixed salt.

Better boundary

Never log the entered password.

Problem / risk

Encryption: designed to be reversed with a key.

*Login does not recover the password.
Login derives a new candidate and compares it.*

Better boundary

Password hashing: one-way verification.

Password: correct-horse-battery-staple

First registration -> random salt A -> derived hash A

Second registration -> random salt B -> derived hash B

Both records verify the same entered password.

The salt is stored; it is not secret.

```
PBKDF2-SHA256$210000$base64-salt$base64-derived-hash
```

part 1: algorithm label

part 2: number of iterations

part 3: random salt

part 4: derived hash

```
byte[] salt = RandomNumberGenerator.GetBytes(16);  
byte[] hash = Rfc2898DeriveBytes.Pbkdf2(  
    password, salt, 210_000,  
    HashAlgorithmName.SHA256, 32);  
  
return algorithm + "$" + iterations + "$" +  
    Convert.ToBase64String(salt) + "$" +  
    Convert.ToBase64String(hash);
```



```
string[] parts = passwordHash.Split("$");  
byte[] salt = Convert.FromBase64String(parts[2]);  
byte[] storedHash = Convert.FromBase64String(parts[3]);  
byte[] candidateHash = Rfc2898DeriveBytes.Pbkdf2(  
    password, salt, iterations,  
    HashAlgorithmName.SHA256, storedHash.Length);  
return CryptographicOperations.FixedTimeEquals(  
    candidateHash, storedHash);
```

1 Read username and password.

V

2 Find a user by username.

V

3 If no user exists, fail with the same answer as a wrong password.

V

4 If a user exists, verify password against PasswordHash.

V

5 On success, set currentUser for this console session.

```
public List<Expense> GetByUserId(int userId) =>
    expenses.Where(expense => expense.UserId == userId).ToList();

public bool DeleteByIdForUser(int expenseId, int userId)
{
    Expense? expense = expenses.FirstOrDefault(item =>
        item.Id == expenseId && item.UserId == userId);
    if (expense is null) return false;
    expenses.Remove(expense); Save(); return true;
}
```

data/

users.json Id, Username, PasswordHash, CreatedAt

expenses.json Id, UserId, Name, Amount, Category, CreatedAt

The original password appears in neither file.

The files are an educational persistence step before SQL.

```
bool isAmountValid = decimal.TryParse(
    rawAmount, NumberStyles.Number,
    CultureInfo.InvariantCulture, out decimal amount);
if (!isAmountValid)
{
    Console.WriteLine("Amount must be a number, for example 12.50.");
    return;
}
expenseService.AddForUser(currentUser, name, amount, category, out message);
```

```
IUserRepository users = new JsonUserRepository(usersFile);  
IExpenseRepository expenses = new JsonExpenseRepository(expensesFile);  
IPasswordService passwordService = new PasswordService();  
  
var authService = new AuthService(users, passwordService);  
var expenseService = new ExpenseService(expenses);  
new ConsoleMenu(authService, expenseService).Run();
```

1 1. Register Alice. Add Lunch for 12.50.

V

2 2. Log out. Register Bob. Log in as Bob.

V

3 3. Bob lists expenses: the list is empty.

V

4 4. Bob tries Alice's expense id: deletion fails.

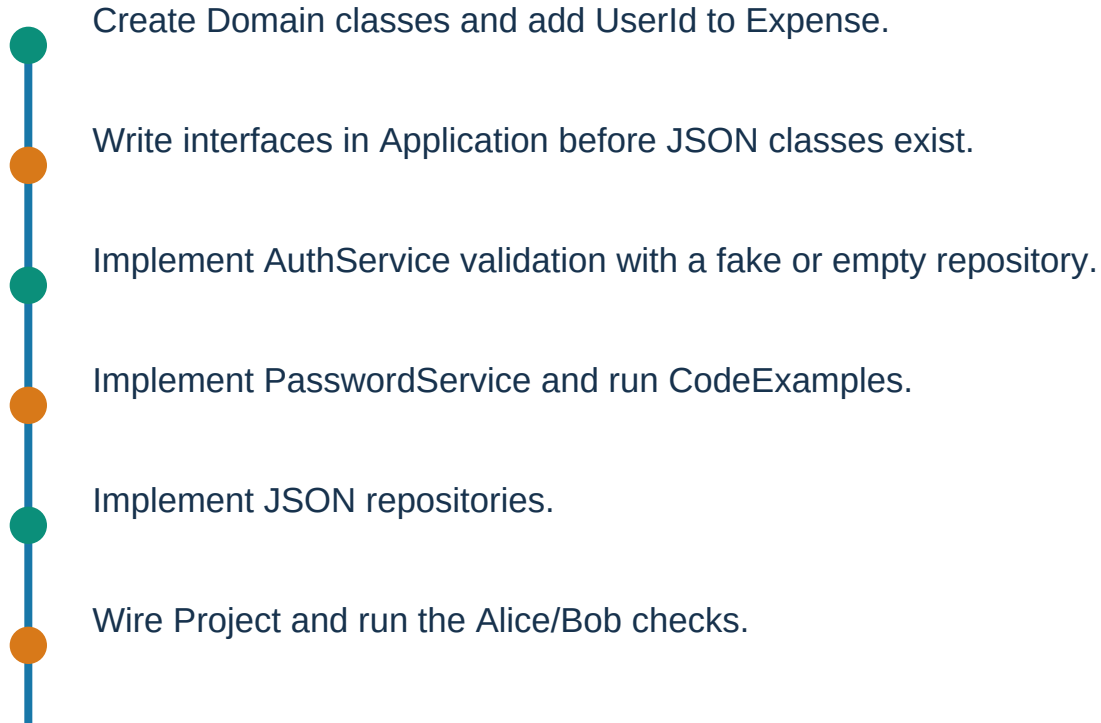
V

5 5. Inspect users.json: PasswordHash exists; Password does not.

V

6 6. Try short password and zero amount: both fail.

Rule in menu only	Move it to a service
Read all data in every caller	Ask repository for user-scoped data
Password in JSON	Store only PBKDF2 record
One fixed salt	Generate random salt per password
Convert with decimal.Parse	Use TryParse at the UI boundary
Specific login error	Use a neutral login error



- Add a maximum expense amount and explain which layer owns the rule.
- Add a Category enum or a category whitelist in ExpenseService.
- Add a change-password use case: verify old password before writing a new hash.
- Add a FindMyExpensesByCategory method that still takes the current user.
- Write a short layer audit: for five lines of code, state why each file owns that line.

Now: User.Id -> Expense.UserId in C# and JSON.

Database course: Users table, Expenses table, primary key, foreign key.

Lecture 09: replace JSON repository internals with SQL/Dapper.

Application services should remain almost unchanged.

That stability is the architectural payoff.

- A solution with the exact Layers, CodeExamples and Project structure.
- Working registration and login.
- PasswordHash stored with PBKDF2, random salt and verification.
- At least one user-owned entity with list and delete operations.
- A short README with the Alice/Bob proof steps and a JSON screenshot with no plain password.

- Why does Application depend on IUserRepository rather than JsonUserRepository?
- Which layer should parse text into decimal?
- Why is a salt stored even though it is not secret?
- What exact condition prevents Bob from deleting Alice's record?
- What is the next swap when the database course reaches SQL?